Carry-less product: `std::clmul`

Document number:
P3642R5

Date:
2026-05-10

Audience:
LEWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-to:
Jan Schultke <janschultke@gmail.com>

GitHub Issue:
wg21.link/P3642/github

Source:
github.com/Eisenwave/cpp-proposals/blob/master/src/clmul.cow

ABSTRACT: Add widening and non-widening carry-less multiplication functions.

Contents

1. Revision history

1.1. Changes since R4

- Rename `std::clmul_wide` to `std::widening_clmul`
- Fix `std::simd::clmul` specified as a unary function (should be binary)
- Rebase §6. Proposed wording on [N5032] with post-Croydon changes applied

1.2. Changes since R3

- Reference faster possible implementations in §4. Possible implementation
- Add some thoughts on why implementations in the compiler are better than pure library implementations
- Mention that `@llvm.clmul` has now been added to LLVM ([LLVMClmul])
- Change `operator<=>` for `wide_result` to be a hidden friend again

1.3. Changes since R2

- Add missing `<T>` for `operator<=>`
- Provide a §4. Possible implementation better suited for SIMD parallelization
- Rebase §6. Proposed wording on N5014
- Improve §6. Proposed wording editorially

1.4. Changes since R1

The paper was seen by SG6 at Sofia 2025 with the following feedback:

> **Summary**: SG6 had no numerics concerns but recommended to include std::simd overloads into the paper.

> **POLL**: Forward P3642R1 to LEWG with the expectation that the next revision includes `std::simd` overloads.
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 7 | 4 | 0 | 0 | 0 |

The following changes were made:

- Provide (non-widening) §5.3. SIMD support
- Use two-space indentation, and generally match the code style of the C++ standard
- Provide detailed design description for §5.2. Widening operation
- Make §6. Proposed wording and design independent of [P3161R4]
- Fix stray `U` type name in §6. Proposed wording, and improve wording generally
- Rebase §6. Proposed wording on `std::simd` naming changes in [P3691R1]

1.5. Changes since R0

- Generate the proposal using COWEL instead of bikeshed
- Fix incorrect formula in §6. Proposed wording for bits ≥ the integer width
- Fix §4.1. Hardware support missing new VPCLMULQDQ instructions
- Fix improper uses of `std::unsigned_integral` in §2. Introduction
- Make slight editorial wording adjustments
- Rebase on N5008 and [P3161R4]
- Mention [SimdJsonClmul] in §3. Motivation

2. Introduction

Carry-less multiplication is a simple numerical operation on unsigned integers. It can be a seen as a regular multiplication where `xor` is being used as a reduction instead of `+`.

It is also known as "XOR multiplication" and "polynomial multiplication". The latter name is used because mathematically, it is equivalent to performing a multiplication of two polynomials in GF(2), where each bit is a coefficient.

I propose a `std::clmul` function to perform this operation:

```
template<class T>
  constexpr T clmul(T x, T y) noexcept;
```

I also propose a widening operation in the style of [P3161R4], as follows:

```
template<class T>
struct wide_result {
```

```
  T low_bits;
  T high_bits;
  // ...
};

template<class T>
  constexpr wide_result<T> widening_clmul(T x, T y) noexcept;
```

3. Motivation

Carry-less multiplication is an important operation in a number of use cases:

- **CRC Computation:** While cyclic redundancy checks can theoretically be performed with a finite field of any length, in practice, GF(2)[X], the *polynomial ring* over the *Galois field* with two elements is used. Polynomial addition in this ring can be implemented via `xor`, and multiplication via `clmul`, which makes cyclic redundancy checks considerably faster.
- **Cryptography:** `clmul` may be used to implement AES-GCM. [IntelClmul] describes this process in great detail and motivates hardware support for carry-less multiplication via the `pclmulqdq` instruction.
- **Bit manipulation:** `clmul` performs a large amount of `<<` and `xor` operations in parallel. This is utilized in the reference implementation [BitPermutations] of `std::bit_compressr`, proposed in [P3104R3]. For example, the form `clmul(x, -1u)` computes the bitwise inclusive parity for each bit of `x` and the bits to its right.

Carry-less multiplication is of such great utility that there is widespread hardware support, some dating back more than a decade. See below for motivating examples.

3.1. Parity computation and JSON parsing

The *parity* of an integer `x` is `0` if the number of one-bits is even, and `1` if it is odd. The parity can also be computed with `popcount(x) & 1`.

EXAMPLE: The special form `clmul(x, -1)` computes the parity of each bit in `x` and the bits to its right. The most significant bit holds the parity of `x` as a whole.

```
bool parity(std::uint32_t x) {
  return std::clmul(x, -1u) >> 31;
}
```

While the parity of *all* bits can be obtained with `clmul`, it computes the inclusive cumulative parity, which can be used to accelerate parsing JSON and other file formats ([SimdJsonClmul]). This can be done by mapping each `"` character onto a `1`-bit, and any other character onto `0`. `clmul(x, -1)` would then produce masks where string characters corresponds to a `1`-bit.

EXAMPLE:

```
abc xxx "foobar" zzz "a" // input string
0000000010000000100000101 // quote_mask
00000000.111111.00000.1. // clmul(quote_mask, -1), ignoring 1-bits of qu
```

§ 3.2. Fast space-filling curves

The special form `clmul(x, -1)` can be used to accelerate the computation of Hilbert curves. To properly understand this example, I will explain the basic notion of space-filling curves.

We can fill space using a 2D curve by mapping the index `i` on the curve onto Cartesian coordinates `x` and `y`. A naive curve that fills a 4x4 square can be computed as follows:

```
struct pos { uint32_t x, y; };

pos naive_curve(uint32_t i) { return { i % 4, i / 4 }; }
```

When mapping the index `i = 0, 1, ..., 0xf` onto the returned 2D coordinates, we obtain the following pattern:

```
0 1 2 3
4 5 6 7
8 9 a b
c d e f
```

The problem with such a naive curve is that adjacent indices can be positioned very far apart (the distance increases with row length). For image processing, if we store pixels in this pattern, cache locality is bad; two adjacent pixels can be very far apart in memory.

A Hilbert curve is a family of space-filling curves where the distance between two adjacent elements is 1:

```
0 1 e f
3 2 d c
4 7 8 b
5 6 9 a
```

De-interleaving bits of `i` into `x` and `y` yields a Z-order curve, and performing further transformations yields a Hilbert curve.

EXAMPLE: `clmul` can be used to compute the bitwise parity for each bit and the bits to its right, which is helpful for computing Hilbert curves. Note that the following example uses the `std::bit_compress` function from [P3104R3], which may also be accelerated using `clmul`.

```
pos hilbert_to_xy(uint32_t i) {
  // De-interleave the bits of i.
  uint32_t i0 = std::bit_compress(i, 0x55555555u); // abcdefgh → bdfh
  uint32_t i1 = std::bit_compress(i, 0xaaaaaaaau); // abcdefgh → aceg
```

```
  // Undo the permutation that Hilbert curves apply on top of Z-order cu
  uint32_t A = i0 & i1;
  uint32_t B = i0 ^ i1 ^ 0xffffu;
  uint32_t C = std::clmul(A, -1u) >> 16;
  uint32_t D = std::clmul(B, -1u) >> 16;

  uint32_t a = C ^ (i0 & D);
  return { .x = a ^ i1, .y = a ^ i0 ^ i1 };
}
```

This specific example is taken from [FastHilbertCurves]. [HackersDelight] explains the basis behind this computation of Hilbert curves using bitwise operations.

When working with space-filling curves, the inverse operation is also common: mapping the Cartesian coordinates onto an index on the curve. In the case of Z-order curves aka. Morton curves, this can be done by simply interleaving the bits of x and y. A Z-order curve is laid out as follows:

```
0 1 4 5
2 3 6 7
8 9 c d
a b e f
```

EXAMPLE: `clmul` can be used to implement bit-interleaving in order to generate a Z-order curves.

```
uint32_t xy_to_morton(uint32_t x, uint32_t y) {
  uint32_t lo = std::clmul(x, x) << 0; // abcd -> 0a0b0c0d
  uint32_t hi = std::clmul(y, y) << 1; // abcd -> a0b0c0d0
  return hi | lo;
}
```

NOTE: In the example above, `std::clmul(x, x)` is equivalent to [P3104R3]'s `std::bit_expand(x, 0x55555555u)`.

4. Possible implementation

A naive and unconstrained implementation looks as follows:

```
template<class T>
  constexpr T clmul(const T x, const T y) noexcept {
    T result = 0;
    for (int i = 0; i < numeric_limits<T>::digits; ++i) {
      result ^= x * (y & (T{1} << i));
    }
```

```
    return result;
}
```

NOTE: This implementation is particularly suited for auto-vectorization. Assuming the loop is unrolled, `T{1} << i` is constant and each bitwise AND and multiplication can be done in parallel. Lastly, all results have to be accumulated using a horizontal XOR.

Expressed in `std::simd` terms, this looks something like:

```
using V = simd::vec<T, numeric_limits<T>::digits>;
static constexpr V powers = array<T, V::size()>{ 0, 1, 2, 4, 8, /* ... */
return simd::reduce(V(x) * (V(y) & powers), bit_xor<T>{});
```

Such a naive implementation is far from optimal though. [QuickBench] shows that a naive `clmul` implementation which computes both the high and the low bits performs 9.2× worse than an efficient implementation taken from [NTL].

NOTE: The mathematical basis for the [NTL] implementation is described in [FasterMultiplicationInGF2].

Since January 2026, LLVM also provides a portable `@llvm.clmul` intrinsic function ([LLVMClmul]). Ideally, an implementation would simply lower `std::clmul` → `__builtin_clmul` → `@llvm.clmul` → `pclmulqdq`.

The issue with library implementations is that the optimal implementation for `std::clmul` highly depends on the architecture and has interesting mathematical properties that become opaque in the library. For example, if one factor is known to be a power of two, even if the exact value isn't known, normal multiplication can be used, which may be faster. Also, carry-less multiplication is commutative.

§ 4.1. Hardware support

The implementation difficulty lies mostly in utilizing available hardware instructions, not in the naive fallback implementation.

In the following table, let `uN` denote `N`-bit unsigned integer operands, and `×N` denote the amount of operands that are processed in parallel.

Operation	x86_64	ARM	RV64
`clmul u64×4 → u128×4`	`vpclmulqdq`[VPCLMULQDQ]		
`clmul u64×2 → u128×2`	`vpclmulqdq`[VPCLMULQDQ]		
`clmul u64 → u128`	`pclmulqdq`[PCLMULQDQ]	`pmull+pmull2`[Neon]	`clmul+clmulh`[Zbc, Zbkc]

`clmul u64 → u128`
`clmul u64 → u64`
`clmul u8×8 → u16×8`
`clmul u8×8 → u8×8`
	`pclmulqdq`[PCLMULQDQ]	`pmull+pmull2`[Neon]
		`pmull`[Neon]
		`pmull`[Neon]
		`pmul`[Neon]
			`clmul+clmulh`[Zbc, Zbkc]
			`clmul`[Zbc, Zbkc]

EXAMPLE: A limited x86_64 implementation of `widening_clmul` may look as follows:

```
#include <immintrin.h>
#include <cstdint>

wide_result<uint64_t> widening_clmul(uint64_t x, uint64_t y) noexcept {
  __m128i x_128 = _mm_set_epi64x(0, x);
  __m128i y_128 = _mm_set_epi64x(0, y);
  __m128i result_128 = _mm_clmulepi64_si128(x_128, y_128, 0);
  return {
    .low_bits  = uint64_t(_mm_extract_epi64(result_128, 0)),
    .high_bits = uint64_t(_mm_extract_epi64(result_128, 1))
  };
}
```

5. Design considerations

Multiple design choices lean on [P0543R3] and [P3161R4]. Specifically,

- the choice of header `<numeric>`,
- the choice to have a widening operation,
- the `_wide` naming scheme,
- the `wide_result` template, and
- the decision to have a `(T, T)` parameter list.

5.1. Naming

Carry-less multiplication is also commonly called "Galois Field Multiplication" or "Polynomial Multiplication".

The name `clmul` was chosen because it carries no domain-specific connotation, and because it is widespread:

- Intel refers to `PCLMULQDQ` As "Carry-Less Multiplication Quadword" in its manual; see [IntelManual].

- RISC-V refers to `clmul` as carry-less multiplication, and this is obvious from the mnemonic.
- The Wikipedia article ([WikipediaClmul]) for this operation is titled "Carry-less product".
- The portable LLVM intrinsic function ([LLVMClmul]) is named `@llvm.clmul`.

5.2. Widening operation

In addition to the `std::clmul` function template, there exists a `std::widening_clmul` function template:

```
template<class T>
struct wide_result {
  T low_bits;
  T high_bits;
  friend constexpr bool operator==(const wide_result&, const wide_result&) = de
  friend constexpr auto operator<=>(const wide_result& x, const wide_result& y)
    noexcept(noexcept(x.low_bits <=> y.low_bits))
    -> decltype(x.low_bits <=> y.low_bits);
};

template<class T>
  constexpr wide_result<T> widening_clmul(T x, T y) noexcept;
```

Such a widening function is important in a various cryptographic use cases. There is universal §4.1. Hardware support for obtaining all 128 bits of a multiplication for that reason.

Most of the design choices take the design of [P3161R4] and [P4052R0] into consideration:

- The function is named `widening_clmul` to be symmetrical with `std::saturating_mul` from [P4052R0] (merged into C++26). That proposal broadly advocated for a naming scheme leaning on Rust, and the Rust standard library has a `widening_mul` function.
- The result type is deliberately not named `wide_clmul_result` so that future `widening_mul` and other operations can use the same result type, which avoids creating an ever-growing set of equivalent (but distinct) types.
- The `low_bits` appear before the `high_bits`, so that on the more widespread little-endian architectures, the layout of the `struct` is identical to that of an integer, which is slightly better for calling conventions.

However, the comparison operators are a novel invention of this proposal. They are intended to behave as if the comparisons were performed on an integer with twice the width of `T`. These comparisons exists so that the result can be easily compared against expected results in test cases, stored in containers like `std::set`, used out of the box with `std::sort`, etc. There is an obvious and mathematically meaningful ordering of `wide_results`, so it would be strange not to add a comparison operator.

Also, `wide_result` should be a broadly useful vocabulary type which may be instantiated with user-defined numeric types, simply because that seems like a useful side product of this proposal.

NOTE: It would not be possible to simply return an integer with twice the width of the input because it is not guaranteed that such a type exists, especially in the case of `unsigned long long` inputs.

5.3. SIMD support

Upon seeing this proposal at Sofia 2025, SG6 recommended to add SIMD support. This recommendation was provided under the assumption that it would be a simple addition in the style of [P2933R4]. Therefore, this proposal provides non-widening SIMD carry-less multiplication with the following signature:

```
template<class T, class Abi>
  constexpr basic_vec<T, Abi> clmul(const basic_vec<T, Abi>& a,
                                    const basic_vec<T, Abi>& b) noexcept;
```

5.3.1. SIMD widening operations are out of scope

AVX-512 provides a `u64×4 → u128×4` operation, and there is currently no precedent for such widening operations in the SIMD library. Specifically, the `VPCLMULQDQ` instruction ignores one of each `u64×2` pairs, and produces a 128-bit output for each such pair.

It would take *considerable* design and wording effort to standardize this, especially if one wants to expose the full `VPCLMULQDQ` behavior, which allows choosing for each `u64×2` integer pair, which of these integers is multiplied and which is ignored. Procedurally, that design effort should be part of [P3161R4] (which proposes widening operations in general) or some follow-up proposal for SIMD widening operations. Some other SIMD instructions like `PMULUDQ` perform multiple widening multiplications in parallel, in the same style as `VPCLMULQDQ`, while some others compute just the upper bits, like `VPMULHUW`. This is a broad design space.

In conclusion, a proposal for widening SIMD operations *in general* would be well-motivated. For `std::clmul`, designing SIMD widening operations would be scope creep.

6. Proposed wording

The proposed changes are relative to [N5032], with the changes accepted during the 2026-02 Croydon meeting applied.

[version.syn]

Change [version.syn] paragraph 2 as follows:

```
[…]
#define __cpp_lib_clmul 20????L // also in <numeric>
[…]
#define __cpp_lib_simd ~~202506L~~ 20????L // also in <simd>
```

DRAFTING NOTE: We only bump `__cpp_lib_simd` without creating a new SIMD feature test macro because [P2933R4] did the same, and because `__cpp_lib_clmul` can be used to test for the presence of both the scalar and SIMD version.

[numeric.ops]

Add the following declarations to the synopsis in [numeric.ops.overview], immediately following the declarations associated with [numeric.sat]:

```
// [numeric.clmul], carry-less product
template<class T>
struct wide_result {
  T low_bits;
  T high_bits;
  friend constexpr bool operator==(const wide_result&, const wide_result&)
  friend constexpr auto operator<=>(const wide_result& x, const wide_resul
    noexcept(noexcept(x.low_bits <=> y.low_bits))
    -> decltype(x.low_bits <=> y.low_bits);
};

template<class T>
  constexpr wide_result<T> widening_clmul(T x, T y) noexcept;
template<class T>
  constexpr T clmul(T x, T y) noexcept;
```

In subclause [numeric.ops], append a subclause immediately following [numeric.sat]:

Carry-less product [numeric.clmul]

```
constexpr auto operator<=>(const wide_result& x, const wide_result& y)
  noexcept(noexcept(x.low_bits <=> y.low_bits))
  -> decltype(x.low_bits <=> y.low_bits);
```

1 *Returns*: `tie(x.high_bits, x.low_bits) <=> tie(y.high_bits, y.low_bits)`.

```
template<class T>
  constexpr wide_result<T> widening_clmul(T x, T y) noexcept;
```

2 Let:

- $\bigoplus$ be a reduction using the exclusive OR operation ([expr.xor]);
- α_i be the i^{th} least significant bit in the base-2 representation of an integer α;
- N be the width of `T`.

3 *Constraints*: `T` is an unsigned integer type ([basic.fundamental]).

4 *Returns*: A `wide_result<T>` object storing the bits of an integer c, where the value of c_i is given by Formula `?.?`, x is `x`, and y is `y`. The result object is initialized

so that

- `low_bits` stores the N least significant bits of c, and
- `high_bits` stores the subsequent N bits of c.

$$c_i = \bigoplus_{j=0}^{i} x_j y_{i-j}$$ [FORMULA ?.?]

```
template<class T>
  constexpr T clmul(T x, T y) noexcept;
```

5 *Effects*: Equivalent to `widening_clmul(x, y).low_bits`.

WARNING: If the mathematical notation in the block above does not render for you, you are using an old browser with no MathML support. Please open the document in a recent version of Firefox or Chrome.

DRAFTING NOTE: The formula is taken from [IntelClmul], with different variable names, and with no special case for the upper N bits; we can simply treat the integers as mathematical integers with $2N$ width.

See [iterator.concept.wine] for precedent on using N to denote the width of a type.

See [sf.cmath.riemann.zeta] for precedent on wording which includes formulae.

DRAFTING NOTE: The formula above in TeX notation is:

```
c_i = \bigoplus_{j = 0}^i x_i y_{i - j}
```

[simd]

Add the following declarations to the synopsis in [simd.syn]:

```
namespace std::simd {
  […]

  // [simd.clmul], carry-less product
  template<class T, class Abi>
    constexpr basic_vec<T, Abi> clmul(const basic_vec<T, Abi>& a,
                                      const basic_vec<T, Abi>& b) noexcept

  // [simd.math], mathematical functions
  template<math-floating-point V> constexpr deduced-simd-t<V> acos(const V
  […]
}
```

In subclause [simd], append a subclause immediately preceding [simd.math]:

Carry-less product [simd.clmul]

```
template<class T, class Abi>
  constexpr basic_vec<T, Abi> clmul(const basic_vec<T, Abi>& a,
                                    const basic_vec<T, Abi>& b) noexcept;
```

1 *Constraints*: The type `V::value_type` is an unsigned integer type ([basic.fundamental]).

2 *Returns*: A `basic_vec` object where the i^{th} element is initialized to the result of `std::clmul(a[i], b[i])` ([numeric.clmul]) for all i in the range [`0`, `V::size()`).

7. References

[N5032] *Thomas Köppe.* Working Draft Programming Languages — C++ (2025-12-15)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf

[P0543R3] *Jens Maurer.* Saturation arithmetic (2023-07-19)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p0543r3.html

[P3104R3] *Jan Schultke.* Bit permutations (2025-02-11)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3104r3.html

[P3161R4] *Tiago Freire.* Unified integer overflow arithmetic (2025-03-26)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p3161r4.html

[P2933R4] *Daniel Towner, Ruslan Arutyunyan.* Extend <bit> header function with overloads for std::simd (2025-02-13)
https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/p2933r4.html

[P3691R1] *Matthias Kretz et al..* Reconsider naming of the namespace for "std::simd" (2025-06-17)
https://wg21.link/p3691r1

[P4052R0] *Jan Schultke, Corentin Jabot.* Renaming saturation arithmetic functions (2025-03-13)
https://wg21.link/p4052r0

[BitPermutations] *Jan Schultke.* C++26 Bit permutations reference implementation
https://github.com/Eisenwave/cxx26-bit-permutations

[SimdJsonClmul] *Geoff Langdale.* Code Fragment: Finding quote pairs with carry-less multiply (PCLMULQD) (2019-03-06)
https://branchfree.org/2019/03/06/code-fragment-finding-quote-pairs-with-carry-less-multiply-pclmulqdq/

[IntelClmul] *Shay Gueron, Michael E. Kounavis.* Intel® Carry-Less Multiplication Instruction and its Usage for Computing the GCM Mode
https://www.intel.com/content/dam/develop/external/us/en/documents/clmul-wp-rev-2-02-2014-04-20.pdf

[IntelManual] *Intel Corporation.* Intel® 64 and IA-32 Architectures Software Developer's Manual
https://software.intel.com/en-us/download/intel-64-and-ia-32-architectures-sdm-combined-volumes-1-2a-2b-2c-2d-3a-3b-3c-3d-and-4

[HackersDelight] *Henry S. Warren, Jr.* Hacker's Delight, Second Edition
https://doc.lagout.org/security/Hackers'Delight.pdf

[FastHilbertCurves] *rawrunprotected.* 2D Hilbert curves in O(1)
"http://threadlocalmutex.com/?p=188

[WikipediaClmul] *Wikipedia community.* Carry-less product
https://en.wikipedia.org/wiki/Carry-less_product

[LLVMClmul] Documentation for 'llvm.clmul.*' Intrinsic
https://llvm.org/docs/LangRef.html#llvm-clmul-intrinsic

[QuickBench] Benchmark of naive clmul vs. NTL clmul
https://quick-bench.com/q/eG4Q5BR_udnfh4V5f-3d3h3fRHY

[NTL] *Victor Shoup.* NTL GitHub repository
https://github.com/libntl/ntl

[FasterMultiplicationInGF2] *Richard P. Brent, Pierrick Gaudry, Emmanuel Thomé, Paul Zimmermann.* Faster Multiplication in GF(2)[x] (2008-11-07)
https://inria.hal.science/inria-00188261v4/document